



BSc. Software Development Year II

Project Report-CA3

Kobi Bourne & Sarah Crotty
C00249676 & C00309469
09/12/2025

Contents

Project Introduction.....	2
Part 1: Hash Table Application.....	3
(a) What is a Hash Table ?	3
(b) Description of your Hash Table application	3
(c) Table of the Data used under the following headings:	3
(d) Diagram of the Hash Table produced- with collisions highlighted.	4
Part 2: Graph Application	5
(a) Diagram of Map of Tourist Sites:	5
(b) Description of Data Structure used to store your Map and any helper variables	5
(c) Diagram of your Map stored in your chosen Data Structure	6
(d) Minimum Spanning Tree of your Graph:	6
(e) Pseudocode of Algorithms	8
Part 3: Code.....	9
References.....	15

Project Introduction

This project is divided into two main parts, each focusing on a different data structure.

Part 1 looks at hash tables and how they can be used in a login system. Ten usernames were chosen, converted to their ASCII values, and then stored in a hash table of size 20 using a simple hash function (mod 20). Linear probing was used whenever a collision occurred. This part shows how hashing works and how data can be stored quickly and efficiently.

Part 2 focuses on graphs. A map of seven tourist sites was created, with each site connected by walking paths of different distances. The graph is stored using an adjacency matrix, and several operations were written for it, including searching for a site, adding new paths, listing all connected sites, and finding the closest location. The report also includes the pseudocode for these methods, sample outputs, and a Minimum Spanning Tree created using Prim's Algorithm.

The aim of the project is to show how these data structures can be applied to real problems and to demonstrate how they organise data and support different operations.

Part 1: Hash Table Application

(a) What is a Hash Table?

A hash table is a data structure that stores elements.

It provides fast and efficient access to elements and is used for quick insertion, Deletion and Searching/Lookups.

When an element is inserted, the hash function converts it to an integer number, which is then used as the index for storing it in the array.

(b) Description of your Hash Table application

Hash table for a login application containing usernames

Hash table size = 20

Usernames = 10

Collision Strategy = Linear Probing

Usernames: kobi, sarah, ellie, darren, wanda, kade, stan, tillie, hazel, June

(c) Table of the Data used under the following headings:

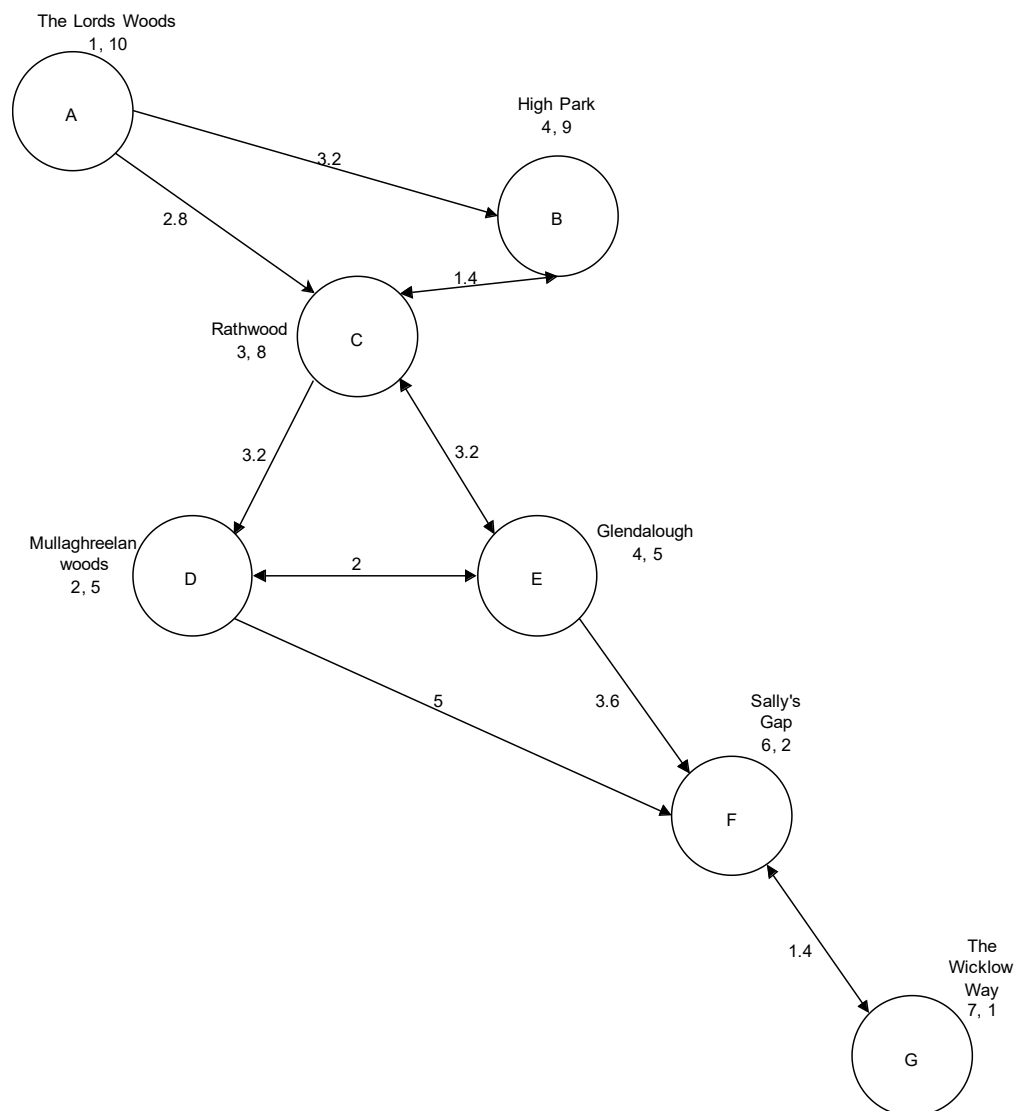
Username	ASCII ADD	ASCII Integer	Index call	Index after Hash function applied
kobi	107+111+98+105	421	421 mod 20	1
sarah	115+97+114+97+104	527	527 mod 20	7
ellie	101+108+108+105+101	523	523 mod 20	3
darren	100+97+114+114+101+110	636	636 mod 20	16
wanda	119+97+110+100+97	523	523 mod 20	3 – 4 (collision)
kade	107+97+100+101	405	405 mod 20	5
stan	115+116+97+110	438	438 mod 20	18
tillie	116+105+108+108+105+101	643	643 mod 20	3-6 (collision)
hazel	104+97+122+101+108	532	532 mod 20	12
june	106+117+110+101	434	434 mod 20	14

(d) Diagram of the Hash Table produced- with collisions highlighted.

Index	Username
0	
1	kobi
2	
3	ellie
4	wanda collision with 3, moved to next free index
5	kade
6	tillie collision with index 3 moved to next free index
7	sarah
8	
9	
10	
11	
12	hazel
13	
14	june
15	
16	darren
17	
18	stan
19	

Part 2: Graph Application

(a) Diagram of Map of Tourist Sites:



The graph used is a weighted, directed graph. It represents connections (walking paths) between 7 vertices (A-G).

Most of the connections are one way, Eg $A \rightarrow B$, however some are directed both ways, Eg $C \leftrightarrow B$.

(b) Description of Data Structure used to store your Map and any helper

variables

The data structure used to store the graph internally is an **Adjacency Matrix**, which is implemented as a **2D-Array**.

Each row represents a start site, and each column represents an end site.

The cells store the weight of the connections/edges. Eg $A \rightarrow B = 3.2$ would be something like `Array[0][1]`.

The helper variables used are as follows: index, row, col -these help navigate the matrix, min and nearest are used for finding the closest site.

(c) Diagram of your Map stored in your chosen Data Structure

This section **MUST** be handwritten (unless you have a handwriting waiver)

Adjacency Matrix

Vertex	A	B	C	D	E	F	G
A	0	3.2	2.8	0	0	0	0
B	0	0	1.4	0	0	0	0
C	0	1.4	0	3.2	3.2	0	0
D	0	0	0	0	2	5	0
E	0	0	3.2	2	0	3.6	0
F	0	0	0	0	0	0	1.4
G	0	0	0	0	0	1.4	0

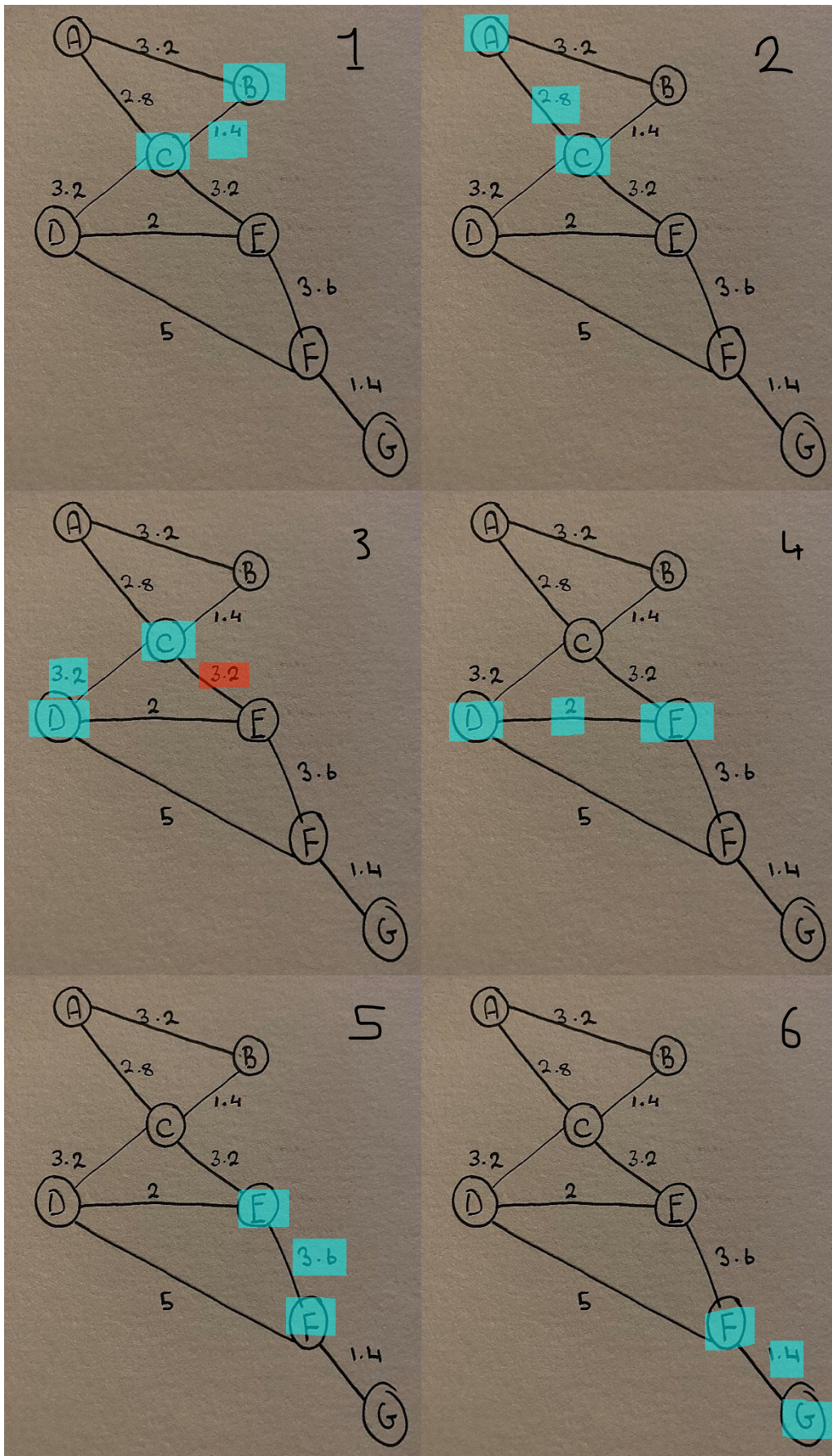
(d) Minimum Spanning Tree of your Graph:

Indicate which algorithm you used to calculate the MST and show the order of the chosen edges.

To calculate the Minimum Spanning Tree, I used Prim's Algorithm.

Prim's Algorithm starts at any chosen node and repeatedly selects the smallest edge that connects a visited node to an unvisited node, until all nodes are included in the MST.

Since MST algorithms only work on undirected graphs, all edges in the graph were treated as undirected to calculate the MST.



Initial Node: C

1. C-B was the shortest edge attached to our initial node at **1.4**.
2. C-A was the next shortest edge attached to our visited nodes at **2.8**.
3. C-D and C-E are both the next shortest at **3.2** I chose C-D.
4. D-E is the next shortest edge attached to our visited nodes at **2**.
5. E-F is the next shortest edge attached to our visited nodes at **3.6**.
6. F-G is the last edge needed to have visited all nodes at **1.4**.

MST: $1.4 + 2.8 + 3.2 + 2 + 3.6 + 1.4 = 14.4$

(e) Pseudocode of Algorithms**Search():**

```
index = getIndex(site)
If (index = -1)
    { return }
output site + " → "
For(col=0; col < sites.length; col++)
    If (matrix[index][col] != 0)
        {Output (sites[col] + " ( " + matrix[index][col] + " ) ")}
```

Insert():

```
row = getIndex(site1)
col = getIndex(site2)
if(row != -1 && col != -1)
    {matrix[row][col] = weight}
```

AllCons():

```
index = getIndex(site)
if(index == -1)
    { return }
Output (site + "connections → ")
for(col=0; col < sites.length; col++)
    {
        if (matrix[index][col] != 0)
            {Output (sites[col])}
    }
```

Closest():

```
index = getIndex(site)
if(index == -1)
    { return }
nearest=-1
min=-1
```

```

        for(col=0; col < sites.length; col++)
        {
            if(matrix[index][col] !=0)
            {
                If (min == -1 || matrix[index][col] < min)
                {
                    min = matrix[index][col]

                    Nearest = col;
                }
            }
        }
        output("The closest site to " + site + "is " + sites[nearest])
    }
}

```

Part 3: Code

A copy of fully documented code

Class Graphs code

```

//Students: Kobi Bourne
//      Sarah Crotty
//Purpose: Class to store graph and its methods
public class Graphs
{
    //declares and initializes Array of site names

    String[] sites = {"The Lords Woods", "High Park", "Rathwood", "Mullaghreeelan Woods", "Glendalough",
    "Sally's Gap", "The Wicklow Way"};

    //declares Adjacency matrix(2-D Array) storing distances

    double[][] matrix =
    {
        {0, 3.2, 2.8, 0, 0, 0, 0},
        {0, 0, 1.4, 0, 0, 0, 0},
        {0, 1.4, 0, 3.2, 3.2, 0, 0},
        {0, 0, 0, 0, 2, 5, 0},
        {0, 0, 3.2, 2, 0, 3.6, 0},
        {0, 0, 0, 0, 0, 0, 1.4},
        {0, 0, 0, 0, 0, 1.4, 0}
    };

    //method to get index of site

```

```

private int getIndex(String site)
{
    for(int index = 0; index < sites.length; index++) //loops through all sites
    {
        if(sites[index].equalsIgnoreCase(site)) //if site = site at current index
        {
            return index; //return site at index
        }
    }
    return -1; //if not found return -1
}

//method to print all leaving connectings for site
public void Search(String site)
{
    int index;

    index = getIndex(site); //get index of site calling method

    if(index == -1) //if site not found
    {
        return; //exit method
    }

    System.out.println(site + " -->"); // print site

    for(int col = 0; col < sites.length; col++) //loops through cloumns in sites row
    {
        if(matrix[index][col] !=0) //if connection
        {
            System.out.println(sites[col] + " ( " + matrix[index][col] + " ) "); //print site and distance
        }
    }
}

```

```

        System.out.println(); // line for spacing

    }

    //add connection
    public void Insert(String site1, String site2, double weight)
    {
        int row;
        int col;

        row = getIndex(site1); //get site1 row
        col = getIndex(site2); //get site2 col

        if(row != -1 && col != -1)
        {
            matrix[row][col] = weight; //add connection, store distance at row,col
        }

        System.out.println("Added new connection");

        System.out.println(); //line for spacing
    }

    //print all connected sites
    public void AllCons(String site)
    {
        int index;

        index = getIndex(site); //get index of site calling method

        if(index == -1) //if site not found
        {
            return; //exit method
        }
    }

```

```

System.out.println(site + " Connections -->"); // print site

for(int col = 0; col < sites.length; col++) //loops through cloumns in sites row
{
    if(matrix[index][col] !=0) //if connection
    {
        System.out.println(sites[col]); //print connected sites
    }
}

System.out.println(); // spacing
}

//print closest site
public void Closest(String site)
{
    int index;
    int nearest;
    double min;

    index = getIndex(site); //get index of site calling method

    if(index == -1) //if site not found
    {
        return; //exit method
    }

    nearest = -1; //closest site index
    min = -1; //smallest distance so far

    for(int col = 0; col < sites.length; col++) //loops through cloumns
    {
        if(matrix[index][col] !=0) //if connection

```

```

        {
            if(min == -1 || matrix[index][col] < min) //if first or smaller
            {
                min = matrix[index][col]; //updates min
                nearest = col; //update nearest
            }
        }
    }

    if(nearest != -1) //if nearest not == -1
    {
        System.out.println("The closest site to " + site + " is " + sites[nearest]); //print nearest site
    }
}

}

```

Class GraphDriver Code

```

//Students: Kobi Bourne
//      Sarah Crotty
//Purpose: Driver to show functionality

public class GraphDriver
{
    public static void main(String [] args)
    {
        Graphs walks = new Graphs(); //new graph object

        walks.Search("The Lords Woods"); //prints connections from the lords woods
        walks.AllCons("Rathwood"); //prints all sites connected to rathwood
        walks.Closest("Sally's Gap"); //prints closest to sally's gap
        walks.Insert("The Lords Woods", "Mullaghreelan Woods", 3.3); //adds connection
        walks.Search("The Lords Woods"); //check added connection
    }
}

```

A list of all the functions/routines which have been used in the code(one line),

getIndex(String site) – returns the position of a site name in the sites array, or -1 if not found.

Search(String site) – prints all outgoing connections and their distances from the given site.

Insert(String site1, String site2, double weight) – adds a edge from site1 to site2 with the given distance in the adjacency matrix.

AllCons(String site) – prints the names of all sites that are directly connected to the given site.

Closest(String site) – finds and prints the closest site directly connected to the given site.

Sample execution screenshots of outputs generated for each operation using the Map/Graph data you provided

walks.Search("The Lords Woods");

```
The Lords Woods -->
High Park ( 3.2 )
Rathwood ( 2.8 )
```

walks.AllCons("Rathwood");

```
Rathwood Connections -->
High Park
Mullaghreelan Woods
Glendalough
```

walks.Closest("Sally's Gap");

```
The closest site to Sally's Gap is The Wicklow Way
```

walks.Insert("The Lords Woods", "Mullaghreelan Woods", 3.3);

```
Added new connection
```

walks.Search("The Lords Woods"); //checking that connection was added

```
The Lords Woods -->
High Park ( 3.2 )
Rathwood ( 2.8 )
Mullaghreelan Woods ( 3.3 )
```

References

Web-site

Author(s) - Bro Code. (2020, October 19th). *Java 2D arrays* 🚚. [Online].
<https://www.youtube.com/watch?v=alwukGslBG8>. Place of publication: Youtube.com.
(03/12/2025)

Author(s) -Alex Lee. (2018, October 25th). *2D Array Java Tutorial #11*. [Online].
<https://www.youtube.com/watch?v=L3-q2GxAqZA>. Place of publication: Youtube.com.
(03/12/2025)

Author(s) - Bro Code. (2021, November 3rd). *Learn Adjacency Matrix in 10 minutes* □. [Online].
<https://www.youtube.com/watch?v=B28xAWEErK8>. Place of publication: Youtube.com.
(05/12/2025)

Author(s)- W3schools. DSA Graphs Implementation. [Online].
https://www.w3schools.com/dsa/dsa_data_graphs_implementation.php. Place of publication: W3schools.com (04/12/2025)

Author(s)- W3schools. Java Multi-Dimensional Arrays. [Online].
https://www.w3schools.com/java/java_arrays_multi.asp. Place of publication: W3schools.com (04/12/2025)

Author(s)-Michael Sambol. Prim's algorithm in 2 minutes. [Online].
<https://www.youtube.com/watch?v=cplfcGZmX7I>. Place of publication: Youtube.com
(06/12/2025)